링크를 만들 때와 클릭할 때

URL 단축기는 짧은 코드와 원래 주소의 연결을 저장하고, 클릭할 때 그 연결을 찾아준다

원래 주소 https://news.example/articles/100

단축 주소 https://short.example/2TX

1. 만들기

긴 URL 입력
POST 요청

→

서버가 연결을 저장
2TX ↔ 원래 URL

→

짧은 링크 반환
이 링크를 공유

2. 사용하기

링크 클릭 → 저장한 원래 주소 조회 → 브라우저가 원래 사이트로 이동

엔드포인트는 요청의 입구다. 단축과 리다이렉션은 각 입구에서 수행하는 기능이다.

POST 요청 안에서 ID와 단축 코드 만들기

이 예시는 “유일 ID를 Base62로 변환하는 방식”을 사용한다

요청 **POST /api/v1/data/shorten**

본문 longUrl = https://news.example/articles/100

먼저 DB 조회: 같은 URL이 이미 있으면 기존 단축 URL을 반환한다.

1. 새 ID 발급
11157 · DB 기본키로 사용

→

2. Base62 변환
11157을 2TX로 표현

→

3. DB에 매핑 저장
저장 완료 후 단축 URL 반환

id	short_code	original_url
11157	2TX	https://news.example/articles/100

ID의 유일성: 각 매핑을 구분 **Base62의 역할: 숫자를 더 짧은 영숫자로 표현**

GET 요청은 저장된 연결을 찾아 이동시킨다

이 예시는 설명을 위해 캐시를 빼고, DB를 직접 조회하는 흐름이다



도메인은 서버를 지정한다. /2TX는 찾을 매핑을 지정한다. 이때 새 ID는 발급하지 않는다.

해시 방식은 단축 코드를 만드는 또 다른 선택지

앞선 Base62 방식과 대안 관계다. 두 방식을 반드시 연달아 수행하는 것은 아니다

해시 후 7글자 추출

원래 URL	→	abc1234	다른 URL과 충돌
원래 URL + #1	→	xyz5678	다시 충돌
원래 URL + #2	→	qtj5opu	사용 가능

추가 문자열은 해시의 입력에 붙인다.

DB에는 원래 URL을 그대로 저장한다.

후보의 존재 검사 비용 줄이기

블룸 필터

“없음”

사전 DB 조회 없이 저장 시도

“있을 수 있음”

오탐 가능하므로 DB에서 확인

필터는 정상적으로 갱신되어 있어야 한다.

원래 URL을 보관하거나 반환하지는 않는다.

블룸 필터는 DB 조회를 줄인다. 동시 저장의 최종 중복 방지는 UNIQUE 등으로 처리한다.

원본은 DB에 보관하고, 반복 조회는 Redis로 줄이기

해시 테이블은 자료구조다. MySQL과 Redis는 앱이 연결해서 쓰는 별도 저장소다

구분	Spring의 HashMap	MySQL	Redis 캐시
어디에 있나?	Java 프로세스의 RAM	DB 프로세스	별도 저장소 프로세스
어떤 역할인가?	앱 안의 메모리 매핑	원본 매핑 영속 보관	코드 → 원래 URL 캐시
앱 서버가 여러 대면?	서버마다 서로 다른 객체	여러 서버가 같은 DB 조회	여러 서버가 같은 Redis 조회



Redis의 값이 만료되어도 DB의 원본 매핑은 남아 있다. 블룸 필터와 역할이 다르다.

7장의 ID 생성기를 가져올 때 확인할 조건

"3,650억 개를 발급할 수 있다"와 "모든 ID가 7글자다"는 서로 다른 조건이다

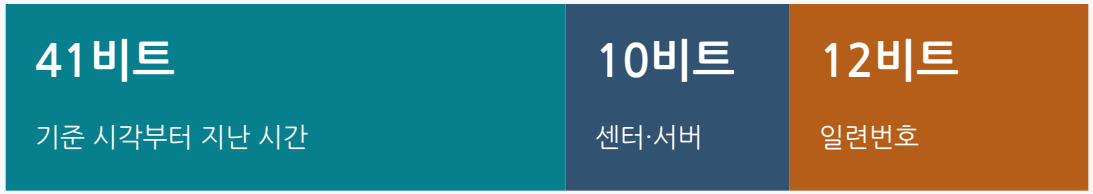
7글자에 들어갈 숫자의 범위

$$0 \le ID < 62^7$$

숫자 7자리: 1,000만 가지

Base62 7자리: 약 3.5조 가지

스노플레이크: 정보를 비트로 조합



부호 1비트는 0. 유일성에는 ID 배정·시계 관리도 필요.

양의 63비트 범위는 Base62로 최대 11글자

유일성 다른 매핑과 겹치지 않는가?

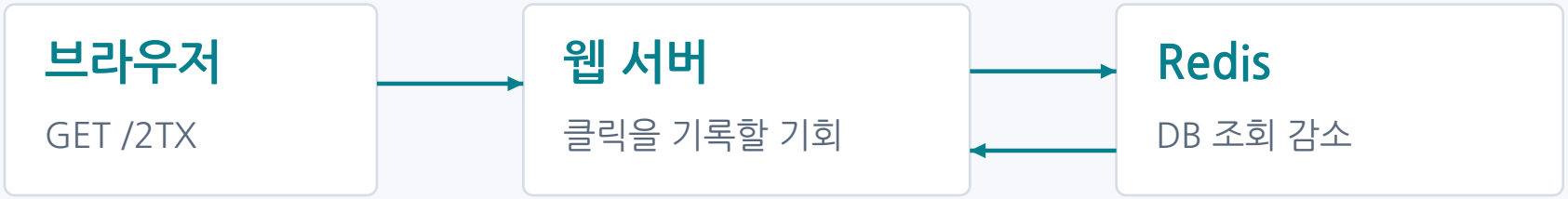
예측 난이도 다른 코드를 추측할 수 있는가?

Base62는 진법 변환이다. 순차 ID나 스노플레이크를 암호처럼 숨겨주지는 않는다.

캐시의 위치가 클릭 분석에 미치는 영향

같은 GET /2TX 요청이라도, 브라우저 캐시와 Redis 캐시는 줄이는 작업이 다르다

서버의 Redis 캐시



웹 서버가 URL을 받아
302 + Location으로 응답

브라우저의 리다이렉션 캐시



301·302의 의미와 함께
Cache-Control도 확인

4장: 생성 요청 제한　　1장: 웹·DB 다중화　　6장: 생성 직후 복제 지연 확인

토론: 클릭 분석이 필요하다면 어느 위치에서 부하를 줄일까? 어떤 요청까지 셀 수 있을까?

문제 1. 역산 불가라면 추측도 불가능할까?

가정: 공개된 해시 함수로 긴 URL을 계산한 뒤 앞 7글자를 코드로 사용한다

개발자의 주장

**“해시값에서 원래 URL을 역산할 수 없으니,
다른 사람이 발급된 단축 코드를 알아낼 수도 없다.”**

추가 조건

상대방은 원래 URL의 유력한 후보 몇 개를 알고 있다.
해시 규칙은 알고 있으며, 비밀 키나 별도 난수는 사용하지 않는다.
이 예시에서는 후보 코드 충돌이 발생하지 않았다고 가정한다.

이 주장은 참일까요? 역산하지 않고도 코드를 찾는 반례가 있을까요?

문제 2. 만료된 코드를 다시 써도 괜찮을까?

책의 기본 설계에 만료·코드 재사용을 추가했다고 가정한다. 두 링크의 문자열은 완전히 같다

과거: 2TX가 문서 A를 가리킴

누군가 링크를 저장함
이후 만료되어 매핑을 삭제

현재: 2TX를 문서 B에 재발급

다른 사람이 새 링크를 받음
주소는 둘 다 short.example/2TX

조건: 로그인·추가 토큰 없이, 서버에 도착한 요청 정보도 같다.

예전 링크를 누른 사람에게는 404, 새 링크를 누른 사람에게는 B로 이동.
302와 no-store를 사용해 브라우저의 이전 이동 캐시는 배제한다.

서버가 두 요청을 구분할 수 있을까요? 어느 조건을 바꿔야 할까요?